



RTL Design Sherpa

Reed-Solomon Codec Hardware Architecture Specification 0.1 (draft)

September 29, 2026

Table of Contents

1 Reed Solomon Has Index.....	9
2 Document Information.....	10
2.1 Reed-Solomon Codec Hardware Architecture Specification.....	10
2.2 Revision History.....	11
2.3 Document Purpose.....	12
2.4 Intended Audience.....	13
2.5 Related Documents.....	14
3 Purpose and Scope.....	15
3.1 Purpose.....	15
3.2 Scope.....	16
3.3 Where this document stands.....	17
4 Document Conventions.....	18
4.1 Terminology.....	18
4.2 Naming.....	19
4.3 Notation.....	20
4.4 Decision markers.....	21
5 Definitions and Acronyms.....	22
6 Use Cases.....	24
6.1 Memory controller ECC.....	25
6.2 Serial link forward error correction.....	26
6.3 Storage erasure coding.....	27
6.4 Standalone codec on a fabric port.....	28

6.5 Memory-to-memory job engine.....	29
7 Key Features.....	30
7.1 Decided (PRD section 3).....	30
7.2 Configurable, not yet chosen (chapter 6.2).....	32
7.3 Explicitly not features.....	33
8 System Context.....	34
8.1 An endpoint codec, not a mid-stream insert.....	34
8.1.1 Figure 2.1: Where the codec sits.....	34
8.2 The core inside a consumer.....	35
8.3 The standalone tops.....	36
8.4 Dependencies on the rest of the repository.....	37
9 Block Diagram.....	38
9.0.1 Figure 3.1: Codec block diagram.....	38
9.1 The encoder core.....	39
9.2 The decoder core.....	40
9.3 The GF layer.....	41
9.4 The adapters.....	42
9.5 Hierarchy.....	43
10 Data Flow.....	44
10.1 Encoder.....	44
10.2 Decoder.....	45
10.2.1 Figure 3.2: Decoder timeline, one symbol per cycle.....	45
10.3 Block boundaries.....	46
10.4 Multiple symbols per beat.....	47

11 Solver Options.....	48
12 Core Interface.....	50
12.1 Clock and reset.....	51
12.2 rs_encoder_core.....	52
12.3 rs_decoder_core.....	54
12.4 Timing rules.....	56
12.5 Erasures (ENABLE_ERASURES, PRD D5 TBD).....	57
13 AXI-Stream Adapter.....	58
13.1 Parameters.....	59
13.2 Behaviour.....	60
14 AXI4 Job Adapter.....	61
14.1 Job interface.....	62
14.2 AXI4 master ports.....	63
14.3 Parameters.....	64
14.4 Behaviour.....	65
15 Register Block.....	66
16 Clock and Reset.....	68
16.1 Single clock.....	68
16.2 Reset.....	69
16.3 Clock gating.....	70
16.4 Frequency.....	71
17 Throughput.....	72
17.1 Sustained rate.....	73
17.2 What scales with S.....	74

18 Latency.....	75
18.1 Encoder.....	76
18.2 Decoder.....	77
18.3 Buffer sizing.....	78
19 Resources.....	79
19.1 GF primitives per core, reference profile.....	80
19.2 Storage.....	81
19.3 Order of magnitude.....	82
19.4 Scaling.....	83
20 System Requirements.....	84
20.1 The consumer provides.....	85
20.2 The consumer gets.....	86
20.3 Environment.....	87
21 Parameter Configuration.....	88
21.1 Code.....	89
21.2 Interface.....	91
21.3 Open decisions in one place.....	92
22 Candidate Profiles.....	93
23 Verification Strategy.....	95
23.1 Golden model.....	95
23.2 Test matrix.....	96
23.3 Test structure.....	97
23.4 Formal.....	98
23.5 Coverage.....	99

24 Synthesis and Implementation.....	100
24.1 Characterisation plan.....	100
24.2 Implementation notes.....	101

List of Figures

Figure 2.1: Where the codec sits.....	33
Figure 3.1: Codec block diagram.....	37
Figure 3.2: Decoder timeline, one symbol per cycle.....	44

List of Tables

Table 1: Table 0.1: Related documents.....	14
Table 2: Table 1.1: Definitions and acronyms.....	22
Table 3: Table 2.1: Use cases and the profile and adapters each implies.....	29
Table 4: Table 2.2: Repository dependencies.....	36
Table 5: Table 3.1: The two key-equation solvers.....	47
Table 6: Table 4.1: Clock and reset.....	50
Table 7: Table 4.2: Encoder core ports.....	51
Table 8: Table 4.3: Decoder core ports.....	53
Table 9: Table 4.4: AXI-Stream mapping.....	57
Table 10: Table 4.5: AXI-Stream adapter parameters.....	58
Table 11: Table 4.6: Job fields.....	61
Table 12: Table 4.7: AXI4 job adapter parameters.....	63
Table 13: Table 4.8: Register block outline.....	65
Table 14: Table 5.1: Sustained throughput, $S = 1$	72
Table 15: Table 5.2: Scaling with symbols per beat.....	73
Table 16: Table 5.3: Encoder latency.....	75
Table 17: Table 5.4: Decoder latency.....	76
Table 18: Table 5.5: GF primitive counts, RS(255,239), $t = 8$, $m = 8$, $S = 1$	79
Table 19: Table 5.6: Storage.....	80
Table 20: Table 6.1: Code parameters.....	88
Table 21: Table 6.2: Interface parameters.....	90
Table 22: Table 6.3: Open PRD decisions and their defaults.....	91
Table 23: Table 6.4: Candidate profiles.....	92
Table 24: Table 6.5: Verification matrix.....	95
Table 25: Table 6.6: Characterisation steps.....	99

1 Reed Solomon Has Index

Generated: 2026-09-29

RTL Design Sherpa · Learning Hardware Design Through Practice [GitHub](#) · [Documentation Index](#) · [MIT License](#)

2 Document Information

2.1 Reed-Solomon Codec Hardware Architecture Specification

Property	Value
Document Title	Reed-Solomon Codec Hardware Architecture Specification
Version	0.1 (draft)
Date	September 29, 2026
Status	Draft – decisions D2, D3, D5 and D10 of the PRD are open; this document carries them as parameters
Classification	Open Source - MIT License

2.2 Revision History

Version	Date	Author	Description
0.1	2026-09-29	RTL Design Sherpa	First draft, written before any RTL exists. Captures what the PRD has decided (valid/ready core with optional AXIS and AXI4 adapters, symbol width as a parameter with the bus a multiple of it, riBM solver with a Euclidean alternative, scrambler behind an enable, endpoint placement, BCH out of scope) and carries the undecided items – correctable-symbol count and profile, shortening, erasures, first consumer – as parameters with candidate values. Numbers are analytic, from the FUB catalog; none are measured.

2.3 Document Purpose

This Hardware Architecture Specification (HAS) is the high-level view of the Reed-Solomon codec – what it is, where it sits, what it presents at its boundaries, what it costs and how it is configured. It is the document an integrator reads before instantiating the core in a memory controller, a compute engine or a link, and the document a verification engineer reads to know what must be proven.

It covers:

- the encoder and decoder cores and their valid/ready interfaces
- the optional AXI-Stream and AXI4 adapters for standalone use
- the configuration parameters and the candidate code profiles
- throughput, latency and resource estimates and how they scale
- integration requirements, verification strategy and synthesis plan

Micro-architecture – the GF arithmetic, the riBM and Euclidean arrays, the Chien and Forney datapaths, FSMs and signal-level timing – belongs to the companion Micro-Architecture Specification (MAS), which does not exist yet. Until it does, `docs/rs_fub_catalog.md` is the block-level reference.

2.4 Intended Audience

- System architects deciding where error correction sits in a datapath
- Integrators instantiating the core or one of its adapters
- Verification engineers planning the golden-model comparison
- Anyone judging whether an RS code, and which one, fits a channel

2.5 Related Documents

Table 0.1: Related documents

Document	Location	Content
Product Requirements	../../PRD.md	the decision table (D1-D12) and the candidate profiles
FUB catalog	../rs_fub_catalog.md	every block bottom-up with what it instantiates
Architecture sketch	../rs_architecture_sketch.md	block diagram and reuse map
References	../../References/README.md	the standards and papers, with source and licence
Handbook	vault/handbook/design/	valid/ready contract, reset and clocking, filelists, generated-RTL discipline

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

3 Purpose and Scope

3.1 Purpose

The Reed-Solomon codec adds symbol-level forward error correction to a data path. An encoder appends $2t$ parity symbols to every block of k data symbols; a decoder, given the $n = k + 2t$ received symbols, corrects up to t symbol errors – or e errors and f erasures with $2e + f \leq 2t$ – and reports what it did. A symbol is m bits, so a single corrupted symbol may carry m corrupted bits: RS is the code for channels whose errors come in bursts and for memories whose failures take out a whole device or a whole nibble at once.

The component exists so that a Sherpa consumer – a memory controller, a compute engine, a serial link – can drop a proven codec into its own datapath instead of building one. That is why the deliverable is a CORE with plain valid/ready streaming ports at both ends (PRD D9): the consumer keeps its own interfaces and the codec is a block inside them.

3.2 Scope

In scope:

- `rs_encoder_core` and `rs_decoder_core`: systematic RS(n, k) over GF(2^m), parameterised in m, t and shortening
- the GF(2^m) primitive layer they are built from, written so the planned BCH component can reuse it
- optional boundary adapters for standalone use: AXI-Stream at either end, or an AXI4 read or write engine with a job interface at either end, chosen independently per end
- an optional scrambler / pseudo-randomizer stage behind a parameter
- the configuration registers, status and counters of the standalone tops
- verification against a software golden model, and FPGA characterisation

Out of scope (PRD section 6 and D7):

- BCH: its own component, `ecc-ip/bch/`
- soft-decision, list and folded RS decoding
- concatenation with convolutional or LDPC codes, and interleaving beyond a symbol-address permutation at the boundary
- any mid-stream insertion of the codec into a fabric (chapter 2.3 says why)

3.3 Where this document stands

No RTL exists at revision 0.1. Everything here is derived from the PRD's decisions, the FUB catalog's block hierarchy and the standards in References/. Where a number appears it is analytic – a cell count from the algorithm, a latency from the pipeline – and is labelled so. Measured throughput, timing and resources arrive with the first synthesis run (chapter 6.5) and will replace the estimates in chapter 5.

RTL Design Sherpa · Learning Hardware Design Through Practice [GitHub](#) · [Documentation Index](#) · [MIT License](#)

4 Document Conventions

4.1 Terminology

- **Symbol:** an m -bit field element. $\text{SYMBOL_WIDTH} = m$. All arithmetic is in $\text{GF}(2^m)$.
- **Block:** n symbols on the coded side, k on the data side. The core delimits blocks with a last flag on the final symbol of each.
- **Beat:** one transfer on a valid/ready port; carries $\text{SYMBOLS_PER_BEAT} = \text{DATA_WIDTH} / \text{SYMBOL_WIDTH}$ symbols.
- **Profile:** one fully-resolved parameter set – m , t , n , k , first root, primitive polynomial, scrambler – typically named after the standard it matches (chapter 6.3).
- **Core:** `rs_encoder_core` or `rs_decoder_core`, valid/ready at both ends.
- **Adapter:** the optional AXIS or AXI4 boundary around a core.

4.2 Naming

Module and signal names follow the repository conventions (vault/handbook/design/): `aclk` / `aresetn`, active-low asynchronous reset through the `ALWAYS_FF_RST` / `RST_ASSERTED` macros, `r_` registers and `w_` wires internally, `valid`/`ready` per the house contract. Parameters are upper case. Generated files carry a “Generated by” banner and live in `rtl/generated/`.

4.3 Notation

- n, k, t, m as above; $2t = n - k$ parity symbols.
- α : the primitive element of $GF(2^m)$ defined by the primitive polynomial $p(x)$; b : the first root of the generator polynomial $g(x) = (x + \alpha^b)(x + \alpha^{b+1}) \dots (x + \alpha^{b+2t-1})$.
- $\Lambda(x)$: error-locator polynomial; $\Omega(x)$: error-evaluator polynomial; S_i : syndromes.
- Counts quoted “for the reference profile” mean RS(255,239), $t = 8$, $m = 8$, one symbol per beat, unless a table says otherwise.

4.4 Decision markers

Where the PRD has decided something, this document states it as fact and cites the decision (for example “PRD D11”). Where the PRD has not, the text says **TBD** and names the decision; chapter 6.2 collects every TBD in one table.

RTL Design Sherpa · Learning Hardware Design Through Practice [GitHub](#) · [Documentation Index](#) · [MIT License](#)

5 Definitions and Acronyms

Table 1.1: Definitions and acronyms

Term	Meaning
RS(n, k)	Reed-Solomon code with n coded symbols per block, k of them data
GF(2 ^m)	the Galois field with 2 ^m elements; symbols and all codec arithmetic live in it
t	the number of symbol errors the code corrects per block; 2t = n - k
erasure	a symbol whose position is known to be unreliable (flagged at the input); costs one parity symbol to fix instead of two
shortened code	RS(n', k') with n' < 2 ^m - 1, derived from the full-length code by treating the missing leading symbols as zero (DVB's RS(204,188) from RS(255,239))
systematic	the k data symbols pass through unchanged and the 2t parity symbols follow them
syndrome	S _i = R(alpha ^(b+i)), the received polynomial evaluated at a generator root; all zero if and only if the block is a codeword
key equation	the relation the solver inverts to find Lambda(x) and Omega(x) from the syndromes
riBM	reformulated inversionless Berlekamp-Massey, the default key-equation solver (Sarwate and Shanbhag 2001)
Euclid / ME	the modified Euclidean algorithm in its inversionless systolic form (Shao et

Term	Meaning
Chien search	al. 1985), the alternative solver evaluating Lambda at every position to find the roots, hence the error locations
Forney	computing each error's magnitude from Omega and Lambda' at the located root
dual basis	the alternative field representation CCSDS uses for its symbols; a fixed linear map at the boundary
scrambler / PRBS	the binary LFSR sequence a standard XORs onto the data for energy dispersal or sync; not part of the RS code
AXIS	AXI4-Stream
PRD	Product Requirements Document (. . . / PRD.md)
MAS	Micro-Architecture Specification (companion, not yet written)
FUB	functional unit block

RTL Design Sherpa · Learning Hardware Design Through Practice [GitHub](#) · [Documentation Index](#) · [MIT License](#)

6 Use Cases

The codec is an endpoint block (chapter 2.3). Its use cases are the endpoints a Sherpa design has or may grow, each matched to one of the PRD's candidate profiles. None is yet a committed consumer – that is PRD decision D10 – so this chapter is the menu the first consumer chooses from.

6.1 Memory controller ECC

A memory controller encodes on its write path and decodes on its read path. Both cores sit inside the controller's own datapath behind its scheduler, which already delimits bursts and so provides the block boundary. Symbols of 8 bits map a byte-wide device failure to one correctable symbol; t is chosen from the device's failure model. This is the shape the pumice / scoria / andesite family would take if it adopted RS over its present SECDED, and it is the use case that argues most strongly for the bare valid/ready core.

6.2 Serial link forward error correction

A transmitter encodes each frame and a receiver decodes it; the scrambler sits outside the encoder on the line side. Two profiles come straight from standards in References/: DVB's RS(204,188) over $GF(2^8)$ and the CCSDS RS(255,223) telemetry code with its dual-basis symbol representation. The 802.3 RS-FEC codes over $GF(2^{10})$ are the high-rate end of this use case and drive the multi-symbol-per-beat option.

6.3 Storage erasure coding

A controller writes parity across devices and reconstructs a lost device on read. This is the erasure-only use of RS (Plank 1997): known-bad positions, no Chien search, any $n \leq 2^m - 1$. It exercises the erasure input (PRD D5) and the AXI4 job adapter (memory-to-memory), and it is the one use case that may want an encoder without a full decoder.

6.4 Standalone codec on a fabric port

The core wrapped in AXI-Stream at both ends, inserted where a stream already has block boundaries (TLAST) and room for the parity – a framer’s output, a DMA’s stream port. This is the AXIS/AXIS adapter pairing and the simplest way to bring the block up in a system.

6.5 Memory-to-memory job engine

The core wrapped in an AXI4 read engine and an AXI4 write engine, driven by jobs from its register block or a descriptor stream: encode this buffer into that one, or decode. A DMA with a transform. This is the AXI4/AXI4 pairing and the natural test vehicle for the erasure use case.

Table 2.1: Use cases and the profile and adapters each implies

Use case	Profile	Adapter pairing	Erasures	Notes
Memory controller ECC	m = 8, t per device model	none (bare cores)	optional	the shape the core was designed for
Serial link FEC	DVB RS(204,188) or CCSDS RS(255,223)	none, or AXIS/AXIS	no	scrambler on the line side
High-rate link FEC	802.3 RS(544,514), m = 10	none	no	needs SYMBOLS_PERR_BEAT > 1
Storage erasure coding	m = 8 or 16, erasure-only	AXI4/AXI4	yes, primary	Plank's construction
Standalone stream codec	any	AXIS/AXIS	optional	bring-up vehicle
Memory-to-memory job	any	AXI4/AXI4	optional	test vehicle for erasures

7 Key Features

7.1 Decided (PRD section 3)

- **Valid/ready core at both ends (D9).** `rs_encoder_core` and `rs_decoder_core` present the house streaming contract: valid, ready, data, a last flag on the final symbol of a block; the decoder adds erasure flags in and a per-block status out. A consumer instantiates the core and nothing else.
- **Optional adapters, selectable per end (D9).** `INTAKE_IF` and `OUTLET_IF` each `NONE` (default), `AXIS` or `AXI4`, independently, so `AXIS-in/AXI4-out` and the other pairings are one core with different adapters generated. Only the adapters selected exist in the netlist.
- **Symbol width is a code property (D1).** `SYMBOL_WIDTH = m` is its own parameter (default 8). `DATA_WIDTH` must be an integer multiple of it, checked at elaboration; `SYMBOLS_PER_BEAT = DATA_WIDTH / SYMBOL_WIDTH` is the throughput factor (D6). A bus that does not divide fails to elaborate rather than straddling symbols across beats.
- **Two key-equation solvers (D11).** `KES_ALGO = "RIBM"` (default): the reformulated inversionless Berlekamp-Massey array, $3t + 1$ cells, shortest critical path. `KES_ALGO = "EUCLID"`: the modified Euclidean array, $2t$ cells, the textbook algorithm in systolic form. Same ports, same result up to a common scale; one generate in the decoder core selects them.
- **Scrambler behind a parameter (D12).** `ENABLE_SCRAMBLER` generates the standard's PRBS stage with its polynomial, seed and placement as parameters; the OFF build carries no LFSR logic and is a tested configuration.
- **Generated field constants (R3).** The primitive polynomial, generator coefficients and log/antilog tables are emitted by a Python generator into `rtl/generated/` from the profile parameters; no field constant is a literal in the datapath.
- **Every block reports (R2).** The decoder never silently passes a failed block: per-block status carries ok / corrected count / uncorrectable, and running counters feed the register block.
- **Golden-model verification (R4).** Every profile is checked against `reedsolo` or `galois` on random data, error patterns up to t and beyond t (to prove the uncorrectable flag), and erasures where enabled.

7.2 Configurable, not yet chosen (chapter 6.2)

- t , the correctable-symbol count, and with it n and k (D2)
- shortening (D3) – full length or a standard's shortened block
- erasure decoding present or absent (D5)
- encoder-only builds for transmit-only or RAID-write consumers (D4)
- per-profile generator conventions: first root b , dual basis (D8)
- the first consumer, which fixes all of the above (D10)

7.3 Explicitly not features

- BCH (its own component), soft-decision or list decoding, interleaving beyond a boundary permutation, concatenation with other codes.

RTL Design Sherpa · Learning Hardware Design Through Practice [GitHub](#) · [Documentation Index](#) · [MIT License](#)

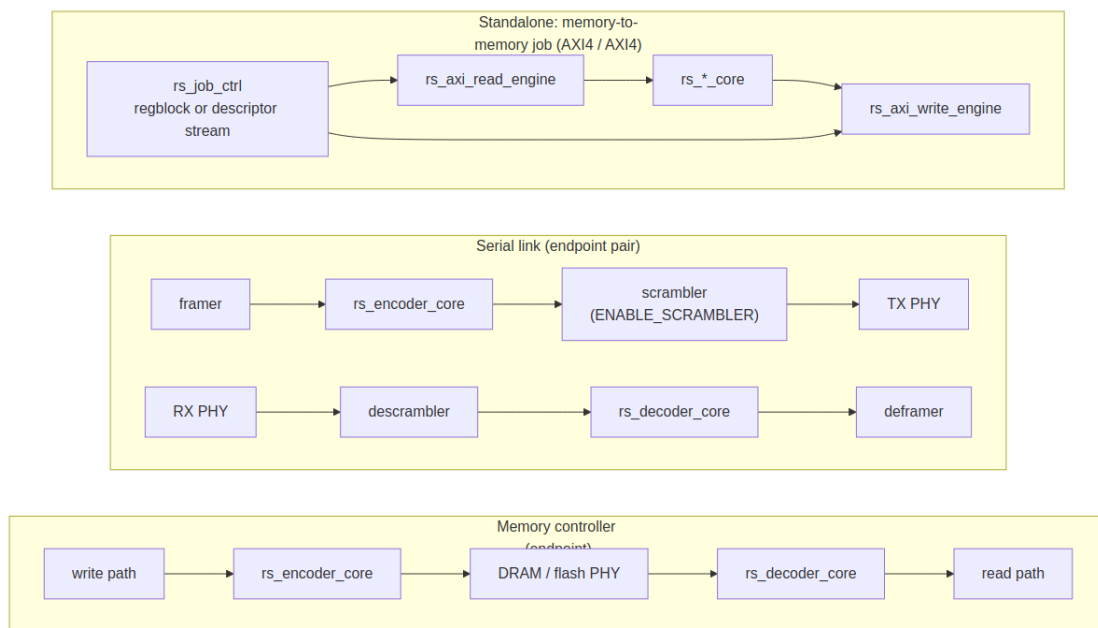
8 System Context

8.1 An endpoint codec, not a mid-stream insert

RS protects a block, not a wire. The encoder must see k symbols before it can emit the first parity symbol, and the decoder must see all n before it can correct any of them, because the syndromes are a function of the whole block. The code therefore lives where a block boundary already exists and where the extra $2t$ symbols have somewhere to go. Every standard in References/ places it at an endpoint: the CCSDS transmitter and ground receiver, the DVB modulator and set-top demodulator, the 802.3 PCS at each end of a link, the CD read channel, the storage controller at write and at read.

Dropping the codec into the middle of a fabric fails on three counts: the parity changes the byte count, so the downstream must run faster or the upstream must leave gaps; block alignment must survive every hop, which is why every RS standard sits behind a sync marker, a fixed frame or a page boundary; and the correction belongs immediately after the channel that corrupts, not several repackings later.

8.1.1 Figure 2.1: Where the codec sits



Where the codec sits

Three endpoint shapes: a memory controller with an encoder on its write path and a decoder on its read path; a serial link pair with the scrambler on the line side; and the standalone memory-to-memory job engine built from the AXI4 adapters.

8.2 The core inside a consumer

With valid/ready at both ends the core is a block in the consumer's own datapath. The consumer supplies block boundaries (its scheduler, framer or job control already knows where they are), absorbs the $2t/k$ rate change, and keeps its own external interfaces. Nothing in the consumer's protocol handling changes; what changes is that k symbols go in and n come out (or n in, k out with a status).

8.3 The standalone tops

When there is no consumer datapath to sit inside – a codec on a fabric port, a memory-to-memory transform – the standalone tops wrap the core in adapters (chapters 4.2 and 4.3) and a register block (chapter 4.4), and the existing AXI-Stream and AXI4 monitors observe the boundaries onto the monitor bus.

8.4 Dependencies on the rest of the repository

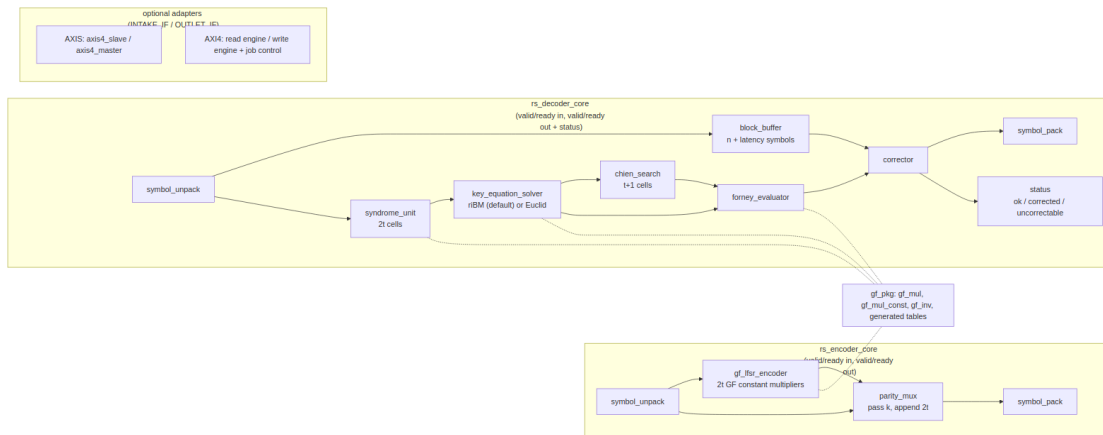
Table 2.2: Repository dependencies

Dependency	Used for
rtl/amba/gaxi: gaxi_skid_buffer, gaxi_fifo_sync	stage decoupling; the decoder's block buffer
rtl/common: counter_bin, shifter_beat_pack, shifter_lfsr_galois	positions and iterations; symbol packing; the scrambler
rtl/amba/axis4: axis4_slave, axis4_master(+ _monlite)	AXIS adapters
rtl/amba/axi4: axi4_master_rd, axi4_master_wr	AXI4 adapters' timing wrappers
rtl/amba/monitor: axis_monitor_lite	boundary observation onto monbus
bin/peakrdl_generate.py + utility-ip/converters APB-to-cpuif path	the register block and its APB attachment
dma-ip/stream/rtl/fub: axi_read_engine, axi_write_engine	the model for the AXI4 job engines
reedsolo, galois (PyPI, MIT)	DV golden model

RTL Design Sherpa · Learning Hardware Design Through Practice [GitHub](#) · [Documentation Index](#) · [MIT License](#)

9 Block Diagram

9.0.1 Figure 3.1: Codec block diagram



Codec block diagram

The two cores, the shared GF layer, and the optional adapters that wrap a core for standalone use.

9.1 The encoder core

Four blocks in a straight line. `symbol_unpack` slices each beat into `SYMBOLS_PER_BEAT` symbols and tracks the position in the block. `gf_lfsr_encoder` is the systematic encoder: a $2t$ -stage shift register over $GF(2^m)$ with the generator polynomial's coefficients as constant multipliers on its taps; it is cleared at block start, the k data symbols shift through, and it then holds the $2t$ parity symbols. `parity_mux` passes the data symbols and then appends the parity, raising `last` on the final one. `symbol_pack` reassembles beats. A skid buffer after the unpack and before the pack decouples the stages.

9.2 The decoder core

The received block is written into `block_buffer` and, in the same cycles, into `syndrome_unit`, whose $2t$ cells each accumulate one syndrome by Horner's rule as the symbols arrive. When the block is complete the syndromes go to `key_equation_solver` – the `riBM` array by default, the modified Euclidean array when selected – which produces the error-locator and error-evaluator polynomials in $2t$ iterations, or flags the block uncorrectable. `chien_search` then walks every position, one per cycle, in lock-step with the buffer read-out; where it finds a root, `forney_evaluator` computes the error value and corrector XORs it into the symbol leaving the buffer. `status` counts corrections and raises the per-block verdict with the final symbol. An all-zero syndrome set bypasses the solver and drains the buffer uncorrected.

9.3 The GF layer

`gf_pkg` holds the field: the primitive polynomial, the generator coefficients and the log/antilog tables, all generated from the profile parameters. Three primitives are built on it: `gf_mul_const` (a constant multiply, an XOR network), `gf_mul` (full multiply, AND array plus reduction) and `gf_inv` (table inverse). Everything arithmetic in both cores is one of these three; the block counts are in chapter 5.3.

9.4 The adapters

Present only when `INTAKE_IF` or `OUTLET_IF` is not `NONE`. The AXI-Stream adapter is the house `axis4_slave` or `axis4_master` wrapper with `TLAST` as the block boundary and `TUSER` carrying erasure flags in or status out. The AXI4 adapter is a read engine (job in, stream out) or a write engine (stream in, bursts out) with a job controller that sequences them from the register block or a descriptor stream. Chapter 4 specifies each.

9.5 Hierarchy

The complete bottom-up hierarchy – every block, what it does and every component it instantiates with counts – is `docs/rs_fub_catalog.md`. This chapter names the blocks; the catalog is the authority on their contents.

RTL Design Sherpa · Learning Hardware Design Through Practice [GitHub](#) · [Documentation Index](#) · [MIT License](#)

10 Data Flow

10.1 Encoder

One beat in, one beat out, with the output stream $2t/k$ longer than the input.

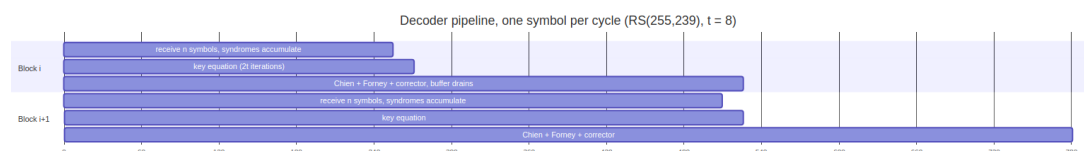
1. A beat of `SYMBOLS_PER_BEAT` data symbols arrives with `in_valid`; the core accepts it when the encoder can take it (`in_ready`).
2. Each symbol enters the LFSR (updating the parity registers) and is passed straight to the output mux – systematic encoding means the data are not delayed by the code, only by the pipeline.
3. On the k -th symbol (`in_last`), the mux switches to the parity registers and emits the $2t$ parity symbols over the following cycles, then raises `out_last`; the LFSR clears for the next block.
4. While parity drains, `in_ready` is low: the encoder cannot start a new block until the previous block's parity has left. A consumer that wants full-rate back-to-back blocks provides an input FIFO of at least $2t$ symbols upstream, or accepts the $2t$ -cycle gap.

10.2 Decoder

One beat in, one beat out, with a block of latency in between.

1. Received symbols are written to the block buffer and fed to the syndrome cells. Erasure flags, if enabled, are recorded alongside each symbol.
2. On `in_last` the syndromes are complete. If all are zero the block is clean: the solver is skipped and the buffer drains through the corrector unchanged, `status = ok`.
3. Otherwise the key-equation solver runs $2t$ iterations. If the resulting locator degree exceeds t (or the Chien search finds fewer roots than the degree), the block is uncorrectable: it drains uncorrected with `status = uncorrectable`, so the consumer sees the block and the verdict together.
4. Chien search steps through positions $0 \dots n-1$, one per cycle, while the buffer reads out the same positions. At each root, Forney supplies the error value and the corrector applies it. `out_last` carries the status.
5. The next block's symbols may be arriving during steps 3-4; the buffer is sized for that overlap, so the decoder sustains one block per n cycles.

10.2.1 Figure 3.2: Decoder timeline, one symbol per cycle



Decoder timeline

For the reference profile a block takes about $2n + 2t$ cycles from first symbol in to last symbol out (about 526 for RS(255,239)), and the next block overlaps the second half, so sustained throughput is one block every n cycles.

10.3 Block boundaries

The core does not count symbols to find a block: `in_last` is the boundary, and the core checks it. A block shorter or longer than the configured `n` (or `k` on the encoder side) is reported as a framing error in status and the core resynchronises on the next `in_last`. A shortened code ($N_SYMBOLS < 2^m - 1$) is handled by the core treating the missing leading symbols as zero in the syndromes and by offsetting the Chien positions; the consumer sees only the shortened block.

10.4 Multiple symbols per beat

With $\text{SYMBOLS_PER_BEAT} = S > 1$ the encoder LFSR advances S symbols per cycle and the syndrome and Chien cells are replicated or fed S -way (chapter 5.1); the solver, which works on the $2t$ syndromes, is unaffected. Blocks are still delimited by `last`; a block whose length is not a multiple of S ends with a partial final beat, marked by a per-symbol keep mask in the data beat.

RTL Design Sherpa · Learning Hardware Design Through Practice [GitHub](#) · [Documentation Index](#) · [MIT License](#)

11 Solver Options

The key-equation solver is the one block with two implementations, selected by KES_ALGO (PRD D11). They have identical ports and produce the same error-locator and error-evaluator polynomials up to a common nonzero scale factor – which is harmless, because Chien search finds the same roots and Forney takes the ratio of the two polynomials.

Table 3.1: The two key-equation solvers

	riBM ("RIBM", default)	modified Euclidean ("EUCLID")
Algorithm	reformulated inversionless Berlekamp-Massey, Sarwate and Shanbhag 2001	Sugiyama's Euclidean algorithm in the inversionless cross-multiplying systolic form of Shao et al. 1985
Array	$3t + 1$ processing elements	$2t$ processing elements
GF multipliers	2 per element: $6t + 2$ (50 at $t = 8$)	4 per element: $8t$ (64 at $t = 8$)
Iterations	exactly $2t$	at most $2t$; may stop early when the remainder degree falls below t
Critical path per iteration	one multiply and one XOR, no feedback across the array	cross-multiply feeding a degree comparison that drives the swap: longer
Control	discrepancy select and gamma update	two degree counters and the swap rule
Readability	dense; the reformulation is not obvious from the textbook	the textbook algorithm, recognisable step by step
Fallback	–	the degree-computationless variant (Baek and Sunwoo 2006) removes the counters at the cost of a wider element

riBM is the default because its per-iteration critical path is shorter and does not cross the array, which is what sets the decoder's clock at high t . Euclid is kept for readability and as the cross-check: both are verified against the same golden model on the same blocks, and the equivalence test is direct – identical Chien root sets and identical Forney values for every corrected block.

Nothing else in the decoder knows which solver produced Lambda. Syndromes, Chien, Forney, the block buffer and the corrector are the same modules in both builds.

RTL Design Sherpa · Learning Hardware Design Through Practice [GitHub](#) · [Documentation Index](#) · [MIT License](#)

12 Core Interface

Both cores present the house valid/ready streaming contract (vault/handbook/design/valid-ready-contracts.md) at each end: valid and data are held stable until ready, ready may depend on valid, valid must not depend on ready. Widths are in bits; S is SYMBOLS_PER_BEAT and m is SYMBOL_WIDTH.

12.1 Clock and reset

Table 4.1: Clock and reset

Port	Dir	Width	Description
aclk	in	1	one clock for the whole core
aresetn	in	1	active-low asynchronous reset, per the repository convention

12.2 rs_encoder_core

Table 4.2: Encoder core ports

Port	Dir	Width	Description
in_valid	in	1	a beat of data symbols is offered
in_ready	out	1	the core takes it this cycle
in_data	in	S x m	S data symbols, symbol 0 in the low bits
in_keep	in	S	which symbols of the beat are present (all ones except possibly the final beat of a block)
in_last	in	1	this beat carries the k-th data symbol of the block
out_valid	out	1	a beat of coded symbols is offered
out_ready	in	1	the consumer takes it
out_data	out	S x m	S coded symbols: the data symbols first, then the 2t parity symbols
out_keep	out	S	present-symbol mask of the beat
out_last	out	1	this beat carries the n-th coded symbol of the block

Port	Dir	Width	Description
frame_err	out	1	pulse: a block ended with other than k data symbols; it was still encoded as given

12.3 rs_decoder_core

Table 4.3: Decoder core ports

Port	Dir	Width	Description
in_valid / in_ready	in / out	1	as the encoder
in_data	in	S x m	S received symbols
in_keep	in	S	present-symbol mask
in_last	in	1	this beat carries the n-th received symbol
in_erase	in	S	(only with ENABLE_ERASURES) symbol is a known erasure
out_valid / out_ready	out / in	1	as the encoder
out_data	out	S x m	S corrected data symbols (parity is not emitted)
out_keep	out	S	present-symbol mask
out_last	out	1	this beat carries the k-th data symbol of the block
out_status_ok	out	1	valid with out_last: the block had no errors
out_status_corre cted	out	$\log_2(t)+1$	valid with out_last: symbols corrected

Port	Dir	Width	Description
out_status_uncor rectable	out	1	(0 .. t) valid with out_last: correction failed; the data are as received
out_status_frame _err	out	1	valid with out_last: block length was not n

12.4 Timing rules

- `in_last` is the block boundary. The core does not infer boundaries by counting; it checks the count against `n` (or `k`) and reports a mismatch.
- The encoder holds `in_ready` low while it drains the $2t$ parity symbols of the previous block (chapter 3.2); a consumer wanting no gap buffers $2t$ symbols upstream.
- The decoder accepts the next block while it is still correcting the previous one, up to the depth of its block buffer; `in_ready` falls only when the buffer is full, which under sustained back-pressure on `out_ready` it will be.
- Status outputs are meaningful only on the beat where `out_last` is high and hold their value until the next block's last beat.
- Reset clears block state and the syndrome and parity registers; the block buffer is not reset (its contents are indexed by pointers that are).

12.5 Erasures (ENABLE_ERASURES, PRD D5 TBD)

When enabled, `in_erase` marks symbols the consumer already knows are bad (a failed device, a dropped packet). Each erasure costs one parity symbol instead of two: the decoder corrects e errors and f erasures with $2e + f \leq 2t$. When disabled the port does not exist and the erasure-locator logic is not generated.

RTL Design Sherpa · Learning Hardware Design Through Practice [GitHub](#) · [Documentation Index](#) · [MIT License](#)

13 AXI-Stream Adapter

Generated at an end when its *_IF parameter is "AXIS". It is the house axis4_slave (intake) or axis4_master (outlet) timing wrapper – the same modules the bridge uses at its AXI-Stream ports – with the core's stream mapped onto the AXI-Stream signals as follows.

Table 4.4: AXI-Stream mapping

AXIS signal	Core signal	Notes
TVALID / TREADY	valid / ready	one-to-one
TDATA[DATA_WIDTH-1:0]	data	S symbols per beat
TKEEP[S*m/8-1:0]	keep	byte keep; symbols are whole bytes when m = 8, else TKEEP is per symbol byte-group and TSTRB is not used
TLAST	last	the block boundary
TUSER (intake, decoder)	in_erase[S-1:0]	erasure flags, only with ENABLE_ERASURES; otherwise unused
TUSER (outlet, decoder)	status	{frame_err, uncorrectable, corrected[...], ok}, valid with TLAST
TID, TDEST	–	passed through unchanged from intake to outlet when both ends are AXIS; tied off otherwise

13.1 Parameters

Table 4.5: AXI-Stream adapter parameters

Parameter	Default	Meaning
AXIS_SKID_DEPTH	2	skid depth in the wrapper (2..8)
AXIS_ID_WIDTH, AXIS_DEST_WIDTH	0	pass-through sideband widths
AXIS_MONITOR	0	1 selects the _monlite wrapper variant, which attaches axis_monitor_lite to the port and emits onto the monitor bus

13.2 Behaviour

The adapter adds no protocol semantics: TLAST is the block boundary the core requires, and the rate change is visible as more beats out than in on the encoder (or fewer on the decoder). A stream that arrives without TLAST at block boundaries cannot be decoded and is reported as a framing error on every block.

[RTL Design Sherpa](#) · [Learning Hardware Design Through Practice](#) [GitHub](#) · [Documentation Index](#) · [MIT License](#)

14 AXI4 Job Adapter

Generated at an end when its *_IF parameter is "AXI4". The intake form is a read engine: a job names a source address and a byte count, the engine issues AXI4 read bursts and presents the returned data as the core's input stream, inserting last every n (decoder) or k (encoder) symbols. The outlet form is a write engine: it drains the core's output stream into AXI4 write bursts at the job's destination address, one block at a time, and reports each block's status in its completion. A job controller sequences the two.

14.1 Job interface

Jobs arrive from the register block (chapter 4.4) – source, destination, count, kick – or, with JOB_IF = "DESC", from a small AXI-Stream descriptor port carrying the same fields for chained jobs.

Table 4.6: Job fields

Field	Width	Meaning
src_addr	AXI_ADDR_WIDTH	read engine start address (intake = AXI4)
dst_addr	AXI_ADDR_WIDTH	write engine start address (outlet = AXI4)
byte_count	32	bytes to read; must be a whole number of blocks
kick / done	1 / 1	start; completion pulse with the job's status summary

14.2 AXI4 master ports

Each engine presents one AXI4 master (read-only or write-only) through the house `axi4_master_rd / axi4_master_wr` timing wrapper: `AXI_ID_WIDTH`, `AXI_ADDR_WIDTH`, `AXI_DATA_WIDTH` (= `DATA_WIDTH`), incrementing bursts up to `cfg_xfer_beats`, no narrow transfers. Responses other than OKAY are counted and reported in the job's done status; the data are still passed.

14.3 Parameters

Table 4.7: AXI4 job adapter parameters

Parameter	Default	Meaning
AXI_ADDR_WIDTH	32	address width
AXI_ID_WIDTH	4	ID width; the engines use one ID
AXI_MAX_BURST_BEATS	16	burst cap (cfg_xfer_beats may lower it at run time)
JOB_IF	"REGS"	"REGS" or "DESC"
AXI_MONITOR	0	1 attaches the _monlite wrapper variant

14.4 Behaviour

The read engine is modelled on STREAM's `axi_read_engine` and the write engine on its `axi_write_engine` – job valid/address/count in, done strobe out, a burst cap – reduced from STREAM's multi-channel, SRAM-coupled form to one job and a FIFO. Whether that is a reduction of STREAM's modules or a fresh instantiation is decided when the adapter is built. The job controller hands the write engine each block's destination before the block's first symbol reaches it, so a decoder job's output blocks land contiguously with the parity removed.

RTL Design Sherpa · Learning Hardware Design Through Practice [GitHub](#) · [Documentation Index](#) · [MIT License](#)

15 Register Block

The standalone tops carry a PeakRDL-generated register block, `rs_regs`, attached through the converters' APB-to-cpuif path exactly as STREAM's configuration block attaches its own. The bare cores have no registers: their configuration is parameters, and their status is the per-block status port. The map below is the outline; the RDL is the authority once written.

Table 4.8: Register block outline

Register	Access	Contents
ID	RO	component ID, version, KES_ALGO, ENABLE_* build flags
PROFILE	RO	m, t, n, k, first root b, primitive polynomial index of the build
CTRL	RW	enable, soft reset, scrambler enable (if built), erasure enable (if built)
JOB_SRC, JOB_DST, JOB_LEN	RW	job fields (AXI4 ends only)
JOB_KICK	WO	start the job
JOB_STATUS	RO	busy, done, last job's status summary, AXI response errors
STAT_BLOCKS	RO	blocks processed
STAT_CORRECTED_BLOCKS	RO	blocks with at least one correction
STAT_CORRECTED_SYMBOLS	RO	total symbols corrected
STAT_UNCORRECTABLE	RO	blocks flagged uncorrectable
STAT_FRAME_ERR	RO	blocks with a length mismatch
IRQ_EN, IRQ_STATUS	RW / W1C	interrupt on uncorrectable, on job

Register	Access	Contents
		done, on frame error

Registers are accessed by name through the generated regmap in DV and host code, never by offset (`vault/handbook/dv/registers-by-name.md`). Counters are 32-bit, saturating, cleared by `CTRL.soft_reset`.

RTL Design Sherpa · Learning Hardware Design Through Practice [GitHub](#) · [Documentation Index](#) · [MIT License](#)

16 Clock and Reset

16.1 Single clock

The core and its adapters run on one clock, `aclk`. A consumer whose data path runs on another clock crosses at the consumer's boundary with the repository's CDC blocks (`rtl/cdc`); the core does not contain a crossing. For a standalone top whose AXI4 memory side runs on a different clock than the stream side, the adapter's timing wrapper is the place to insert the `axi4_cdc_{rd,wr}` pair, as the bridge does for its CDC slave ports; that is a build option for the adapter, not a property of the core.

16.2 Reset

`aresetn` is active-low and asynchronous, applied through the `ALWAYS_FF_RST` macros with `RST_ASSERTED` so the polarity follows the repository build (`vault/handbook/design/reset-and-clocking.md`). Reset clears control state, the parity and syndrome registers, position and iteration counters and the status. The block buffer's storage is not reset; its pointers are, so stale contents are never read.

16.3 Clock gating

None inside the core. The `_cg` variants of the AXIS and AXI4 wrappers may be selected at the adapters for a standalone top that wants boundary clock gating, following the rest of the repository.

16.4 Frequency

The decoder's clock is set by the key-equation solver's per-iteration critical path (one GF multiply plus an XOR for riBM), then by the syndrome and Chien cells (one constant multiply plus an XOR). At $m = 8$ these are a few LUT levels; chapter 5.3 gives the characterisation plan that turns this into a number per part.

RTL Design Sherpa · Learning Hardware Design Through Practice [GitHub](#) · [Documentation Index](#) · [MIT License](#)

17 Throughput

All figures analytic at revision 0.1.

17.1 Sustained rate

Both cores consume one beat per cycle when not back-pressured. With `SYMBOLS_PER_BEAT = S`, that is S symbols, $S \times m$ bits, per cycle.

Table 5.1: Sustained throughput, $S = 1$

Core	Sustained	Gap	Notes
Encoder	k symbols in per k + $2t$ cycles	$2t$ cycles per block while parity drains, unless the consumer buffers $2t$ symbols upstream	output rate is n/k times the input rate
Decoder	one block per n cycles	none once pipelined; the next block arrives during the previous block's Chien walk	output rate is k/n times the input rate

For the reference profile at $S = 1$ and a 200 MHz clock: 1.6 Gbit/s coded on the wire, 1.5 Gbit/s of data. At $S = 8$ (a 64-bit bus, $m = 8$) the same clock gives 12.8 Gbit/s coded, which is where the 802.3-class profiles live.

17.2 What scales with S

Table 5.2: Scaling with symbols per beat

Block	Scaling	Reason
encoder LFSR	S symbols advance per cycle	the LFSR update is applied S times in one cycle (or unrolled)
syndrome cells	S-fold Horner step	each cell absorbs S symbols per cycle
Chien cells	S positions per cycle	replicated, or the step constant becomes $\alpha^{(-S)}$ with S evaluators
Forney	up to S corrections per cycle in the worst case	S inverses, or a small queue and a one-per-cycle Forney with a stall on dense errors
block buffer	width x S	depth unchanged in beats
key-equation solver	unchanged	it works on the 2t syndromes, not the stream

The Forney row is the one to watch: with $S > 1$ a block with errors in adjacent positions may need more than one correction in a cycle. The cheap design stalls the drain for the extra cycles (rare, bounded by t per block); the expensive one replicates the inverse.

RTL Design Sherpa · Learning Hardware Design Through Practice [GitHub](#) · [Documentation Index](#) · [MIT License](#)

18 Latency

All figures analytic at revision 0.1, one symbol per beat.

18.1 Encoder

Table 5.3: Encoder latency

From	To	Cycles
first data symbol in	first data symbol out	pipeline only: about 3 (unpack, skid, pack)
k-th data symbol in	first parity symbol out	pipeline + 1
k-th data symbol in	last parity symbol out	pipeline + 2t

18.2 Decoder

Table 5.4: Decoder latency

Phase	Cycles	Notes
receive the block	n	syndromes complete with the last symbol
key equation	$2t$ (riBM) or $\leq 2t$ (Euclid)	skipped when the syndromes are all zero
Chien + Forney + corrector	n	one position per cycle; the buffer drains in step
pipeline	about 6	unpack, skid, solver handoff, Forney, corrector, pack
first symbol in to last symbol out	about $2n + 2t + 6$	527 for RS(255,239)
first symbol in to first symbol out	about $n + 2t + 6$	277

For a shortened code the n above is the shortened length (204 for DVB). For S symbols per beat, divide the n terms by S; the $2t$ solver term does not shrink.

18.3 Buffer sizing

The block buffer must hold the current block while the next arrives during the Chien walk: depth = $n + 2t + \text{pipeline symbols}$, rounded up to a power of two – 512 for the reference profile, 1024 for RS(544,514).

RTL Design Sherpa · Learning Hardware Design Through Practice [GitHub](#) · [Documentation Index](#) · [MIT License](#)

19 Resources

Analytic counts at revision 0.1, from `docs/rs_fub_catalog.md`; no synthesis has been run. Chapter 6.5 is the plan that replaces this table with measurements.

19.1 GF primitives per core, reference profile

Table 5.5: GF primitive counts, RS(255,239), $t = 8$, $m = 8$, $S = 1$

Primitive	Encoder	Decoder (riBM)	Decoder (Euclid)	Formula
gf_mul_con st	16	33	33	encoder $2t$; decoder $2t$ + $(t+1) + t$
gf_mul	0	52	66	riBM $2(3t+1) + 2$; Euclid $8t +$ 2 ; + t with erasures
gf_inv	0	1	1	one, in Forney
GF symbol registers	16	75	89	encoder $2t$; decoder $2t$ + $(t+1) +$ solver array

19.2 Storage

Table 5.6: Storage

Item	Size	Formula
block buffer	512 x 9 bits (one BRAM)	$(n + 2t + \text{pipe, rounded up}) \times (m + 1 \text{ erasure bit})$
log / antilog tables (gf_inv)	2 x 256 x 8 bits	$2 \times 2^m \times m$; LUT ROM at $m = 8$, BRAM at $m = 10$
skid buffers	2 (encoder), 3 (decoder), 2 deep	stage boundaries

19.3 Order of magnitude

A `gf_mul_const` at $m = 8$ is at most 8 XOR trees of up to 8 inputs; a `gf_mul` is an 8×8 AND array with reduction, on the order of 60-80 LUT6 equivalents. That puts the reference decoder's arithmetic near 4-5 k LUTs on a 7-series part before control and the buffer, and the encoder near 200. The converters-area dwidth blocks and the bridge's monitors give the repository calibration points for what a block of this size does on Artix-7 and Kintex-7; the first out-of-context run will say where the RS decoder lands.

19.4 Scaling

- CCSDS RS(255,223), $t = 16$: every $2t$ -scaled row doubles (32 syndrome cells, 49 riBM elements, 17 Chien cells); plus the dual-basis conversion at both boundaries.
- 802.3 RS(544,514), $m = 10$: every multiplier grows from 8×8 to 10×10 , the tables to 1024 entries, the buffer to 1024 deep; $S > 1$ is a requirement, not an option, at those rates.
- S symbols per beat: syndrome and Chien cells $\times S$; solver unchanged.

RTL Design Sherpa · Learning Hardware Design Through Practice [GitHub](#) · [Documentation Index](#) · [MIT License](#)

20 System Requirements

What a consumer must provide, and what it gets.

20.1 The consumer provides

- **Block boundaries.** `in_last` on the k -th data symbol (encoder) or n -th coded symbol (decoder) of every block. The consumer's scheduler, framer or job control already knows where blocks end; the core checks and reports, it does not infer.
- **Room for the rate change.** The encoder emits n symbols for every k it takes; the decoder emits k for every n . The consumer's downstream (encoder) or upstream (decoder) absorbs the difference, or provides a FIFO of $2t$ symbols so the encoder's parity drain does not stall its input.
- **A single clock** and the repository's active-low asynchronous reset.
- **Erase knowledge**, if `ENABLE_ERASURES`: which symbols are known bad, per symbol, with the data.
- **Configuration at build time.** The profile is parameters; nothing about the code is set at run time in the bare core.

20.2 The consumer gets

- Systematic encoding: data pass through unchanged and in order, parity follows.
- A per-block verdict on the decoder: ok, corrected (with count), uncorrectable, framing error – valid with the last symbol of the block, so the consumer can act on the block it has just received.
- Data as received on an uncorrectable block, never silently altered.
- Determinism: no run-time state survives a block except the counters; the same input block yields the same output block every time.

20.3 Environment

- SystemVerilog-2012, synthesisable, no assertions in RTL (properties live in `formal/`), lint-clean under Verilator per the repository gate.
- Filelists under `rtl/filelists/`, registered in `bin/filelists.toml`, one per top, from the first module.
- Python golden model dependencies (`reedsolo`, `galois`) are DV-only.

RTL Design Sherpa · Learning Hardware Design Through Practice [GitHub](#) · [Documentation Index](#) · [MIT License](#)

21 Parameter Configuration

Every parameter of the two cores and the standalone tops. **TBD** marks a PRD decision not yet taken; the value shown is the working default the reference profile uses.

21.1 Code

Table 6.1: Code parameters

Parameter	Type	Range	Default	Meaning	PRD
SYMBOL_WIDTH	int	3..16	8	m; the field is $GF(2^m)$	D1 decided
T_SYMBOLS	int	$1..(2^m-1)/2$	8 TBD	correctable symbols per block; 2t parity	D2
N_SYMBOLS	int	$2t+1 .. 2^m-1$	255 TBD	block length; $< 2^m-1$ is a shortened code	D3
PRIM_POLY	int	irreducible of degree m	0x11D	primitive polynomial ; selects the field representation	D8 per profile
FIRST_ROOT	int	$0 .. 2^m-2$	0 TBD	b, first root of g(x); DVB 0, CCSDS 112	D8
DUAL_BASIS	bit		0	CCSDS dual-basis symbol representation at the boundaries	D8
ENABLE_ERASURES	bit		0 TBD	erasure input and erasure-locator logic	D5

Parameter	Type	Range	Default	Meaning	PRD
KES_ALGO	string	"RIBM", "EUCLID"	"RIBM"	(decoder) key- equation solver	D11 decided

Derived: $K_SYMBOLS = N_SYMBOLS - 2 * T_SYMBOLS$.

21.2 Interface

Table 6.2: Interface parameters

Parameter	Type	Range	Default	Meaning	PRD
DATA_WIDTH	int	multiple of SYMBOL_WIDTH	8	beat width; elaboration fails if not a multiple	D1 decided
SYMBOLS_PER_BEAT	derived		1	DATA_WIDTH / SYMBOL_WIDTH, the throughput factor	D6
INTAKE_IF	string	"NONE", "AXIS", "AXI4"	"NONE"	intake adapter (standalone tops)	D9 decided
OUTLET_IF	string	"NONE", "AXIS", "AXI4"	"NONE"	outlet adapter	D9 decided
ENABLE_SCRAMBLER	bit		0	line scrambler stage	D12 decided
SCRAMBLER_POLY, SCRAMBLER_SEED, SCRAMBLER_AFTER_ENCODER	int, int, bit		DVB values	the standard's PRBS	D12

Adapter-specific parameters are in tables 4.5 and 4.7.

21.3 Open decisions in one place

Table 6.3: Open PRD decisions and their defaults

Decision	What fixes it	Effect if left at the default
D2 t	the consumer's error model or the standard	RS(255,239)-class: 8 symbol errors per 255
D3 shortening	the consumer's block size	full-length blocks
D4 encoder-only	a transmit-only or RAID-write consumer	both cores are built; a consumer instantiates what it needs
D5 erasures	storage / RAID consumer says yes	no erasure port, smaller decoder
D8 conventions	the standard	b = 0, no dual basis
D10 first consumer	you	none of the above changes until it appears

RTL Design Sherpa · Learning Hardware Design Through Practice [GitHub](#) · [Documentation Index](#) · [MIT License](#)

22 Candidate Profiles

A profile is a resolved parameter set. These four come from the standards and papers in References/; the first consumer picks one or defines its own.

Table 6.4: Candidate profiles

Profile	m	t	n / k	PRIM_P ONLY	FIRST_ ROOT	Scramb ler	Source	Notes
Refere nce (DVB outer code, full length)	8	8	255 / 239	$x^{8+x}4+x^{3+x}2+1$	0	$x^{15+x}14$ +1, seed 100101 010000 000, before the encode r	EN 300 429 / 744 sec. 4.3 (verifi ed)	the profile every count in this docum ent quotes
DVB shorte ned	8	8	204 / 188	same	0	same	same	51 leadin g zero symbo ls droppe d; MPEG- 2 transp ort packet s
CCSDS teleme try	8	16 (or 8)	255 / 223 (239)	$x^{8+x}7+x^{^2+x}+1$ (CCSDS field)	112	$x^{17+x}14$ +1, seed 110001 110001 11000, after	CCSDS 131.0- B-5 sec. 4 and 10	dual- basis symbo l repres entatio n;

Profile	m	t	n / k	PRIM_P ONLY	FIRST_ ROOT_	Scramb ler	Source	Notes
						the encode r (legacy : $x^{8+x}7+x^{5+x}3+1$, all ones)		interle ave depth 1-8 outsid e the core
Ethern et RS- FEC	10	15 (KP4) / 7 (KR4)	544 / 514, 528 / 514	$x^{10+x}3+$ 1	0	802.3 scram bler is separa te	IEEE 802.3 cl. 91 / 108 (cited)	S > 1 requir ed; m = 10 tables
RAID erasur e	8 or 16	any	any n <= 2^m-1	any	–	none	Plank 1997 + 2003	erasur e-only; encode r + erasur e decode r, no Chien search

The DVB row is the one verified line by line against the standard's text (2026-09-29); CCSDS is read from the stored Blue Book; the 802.3 and RAID rows are from the cited documents and should be confirmed against them when chosen.

23 Verification Strategy

23.1 Golden model

Every configuration is verified against a software Reed-Solomon implementation on the same blocks: `reedsolo` (MIT) with the profile's primitive polynomial, first root and shortening, cross-checked by `galois` (MIT) for the field tables the generator emits. The TB encodes with the model and checks the RTL encoder byte for byte; it injects errors into model-encoded blocks and checks the RTL decoder's corrected output and verdict against the model's decode.

23.2 Test matrix

Table 6.5: Verification matrix

Axis	Values	Why
profile	each row of table 6.4 the build admits	field constants and shortening differ per profile
errors per block	0, 1, t-1, t, t+1, random	0 exercises the bypass; t+1 must be flagged uncorrectable, never miscorrected
erasures (if built)	0 .. 2t, mixed with errors up to $2e + f = 2t$	the erasure budget
symbols per beat solver	1, and the consumer's S riBM and Euclid	the S-fold datapaths identical roots and values on every corrected block
scrambler	off and on	the OFF build must be tested as its own configuration
adapters	NONE/NONE first; then AXIS/AXIS and AXI4/AXI4; the mixed pairs follow	the core is proven before any wrapper
back-pressure	none, random on both ends, sustained on the output	the valid/ready contract and the block buffer's full behaviour
framing	short block, long block, missing last	the framing-error path

23.3 Test structure

cocotb tests under `dv/tests/` with TB classes in `dv/tbclasses/`, Pattern B (`cocotb_test_*` prefixes), the three run levels through both the `REG_LEVEL` grid and `TEST_LEVEL` depth, per the `test-patterns` skill. Traffic at the adapters uses the existing AXI-Stream and AXI4 BFM; nothing drives a valid/ready port by hand. Registers are read by name through the generated regmap.

23.4 Formal

No assertions in RTL. Candidate properties for formal/ blocks: the valid/ready contract at both ends of each core; the encoder's parity equals the golden LFSR for one block (bounded); the solver's degree invariant ($\deg \text{Lambda} \leq \text{number of iterations}$) and the equivalence of the two solvers on symbolic syndromes for small t .

23.5 Coverage

Functional coverage on the matrix above plus: every syndrome cell non-zero at least once, every Chien position rooted at least once across the run, Forney with a zero and a maximal error value, the uncorrectable path taken by both the degree check and the root-count check.

RTL Design Sherpa · Learning Hardware Design Through Practice [GitHub](#) · [Documentation Index](#) · [MIT License](#)

24 Synthesis and Implementation

24.1 Characterisation plan

Numbers in chapter 5 are analytic. They are replaced by measurements the way the repository measures every other block: an out-of-context synthesis fixture per configuration, on the same two parts the monitor and bridge characterisations use.

Table 6.6: Characterisation steps

Step	Fixture	Output
1	rs_encoder_core and rs_decoder_core, reference profile, S = 1, Artix-7 100T -1 at 10 ns and Kintex-7 325T -2 at 6.667 ns	LUTs, flops, BRAM, WNS; the calibration for table 5.5
2	decoder with KES_ALGO = "EUCLID"	the solver comparison in table 3.1 measured
3	S = 2, 4, 8	the scaling in table 5.2 measured; the Forney stall design decided
4	CCSDS and 802.3 profiles	t and m scaling
5	standalone tops with each adapter pair	adapter cost; the monitors' cost is already known

The fixture follows `projects/components/fabric-gen-ip/bridge/fpga/` (make `synth BRIDGE=<fixture>` on `make/fpga_flow.mk`) and reports through the same `summary.csv` shape, so the numbers land beside the bridge's and the monitor's.

24.2 Implementation notes

- All arithmetic is combinational XOR/AND networks; there are no carries, so the multiplier trees of rtl/math do not apply and the synthesiser's DSP inference is irrelevant.
- `gf_inv` at $m = 8$ is two 256-entry ROMs and infers to LUTs; at $m = 10$ it becomes BRAM or Itoh-Tsujii.
- The block buffer infers one BRAM at the reference depth; `REGISTERED = 1` on the FIFO adds a cycle and removes the read mux from the corrector's path if timing needs it.
- The riBM array has no cross-array feedback, so its cells may be pipelined per iteration if the clock demands; the Euclid array's degree compare cannot be pipelined without the degree-computationless variant.
- No clock gating in the core; boundary gating comes from the `_cg` wrapper variants at the adapters.